

重庆大学计算机专业核心课程《机器学习》

第2章 模型评估与选择

授课教师：郑林江

第2章 模型评估与选择

➡ 2.1 经验误差与过拟合

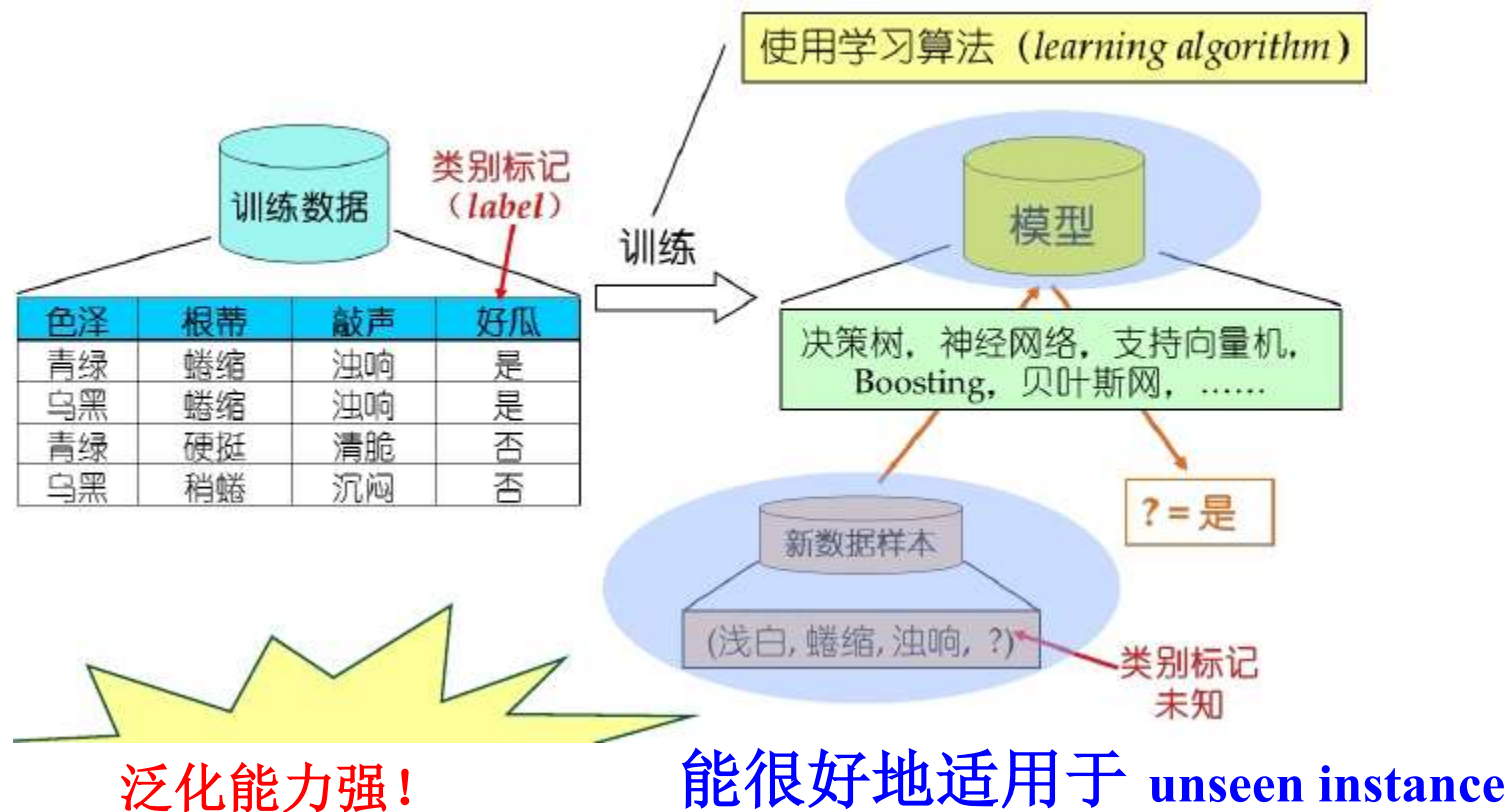
2.2 评估方法

2.3 性能度量

2.4 比较检验

2.5 偏差与方差

□ 典型机器学习过程



例如, 错误率低、精度高

然而, 我们手上没有 unseen instance,

□ 泛化误差 vs. 经验误差

- 泛化误差：在“未来”样本上的误差
- 经验误差：在训练集上的误差，亦称“训练误差”
 - ✓ 泛化误差越小越好
 - ✓ 经验误差是否越小越好？

NO! 因为会出现“过拟合” (overfitting)

■ 错误率&误差:

□ 错误率: 错分样本的占比: $E = a/m$

□ 误差: 样本真实输出与预测输出之间的差异

✓ 训练(经验)误差: 训练集上

✓ 测试误差: 测试集

✓ 泛化误差: 除训练集外所有样本

□ 由于事先并不知道新样本的特征, 我们只能努力使经验误差最小化;

□ 很多时候虽然能在训练集上做到分类错误率为零, 但多数情况下这样的学习器并不好

□ 过拟合:

- ✓ 学习器把训练样本学习的“太好”，将训练样本本身的特点当做所有样本的一般性质，导致泛化性能下降
- ✓ 优化目标加正则项
- ✓ early stop

□ 欠拟合:

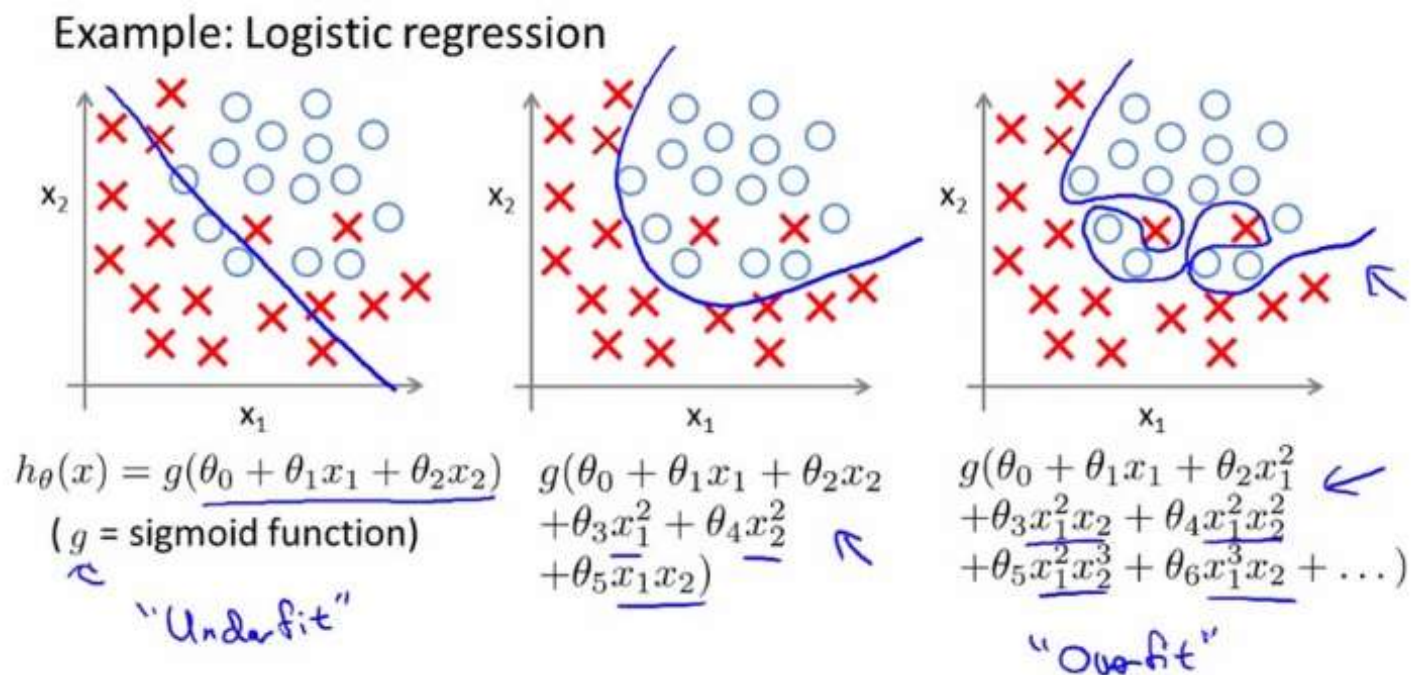
- ✓ 对训练样本的一般性质尚未学好
- ✓ 决策树: 拓展分支
- ✓ 神经网络: 增加训练轮数



过拟合、欠拟合的直观类比

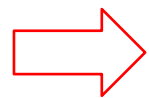
■ 过拟合

- ✓ 模型在训练集模型表现良好，但是在测试集较差。
- ✓ 具体表现在训练集为追求好的效果（**经验损失小，准确率高**等），模型建立的过于复杂，能够很好反映已知数据。
- ✓ 但**泛化能力太差**。即empirical loss比较小，而Generalized loss比较大。



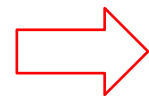
三个关键问题：

■ 如何获得测试结果？



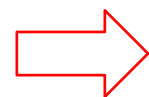
评估方法

■ 如何评估性能优劣？



性能度量

■ 如何判断实质差别？



比较检验

第2章 模型评估与选择

2.1 经验误差与过拟合



2.2 评估方法

2.3 性能度量

2.4 比较检验

2.5 偏差与方差

- ✓ 现实任务中往往会对学习器的泛化性能、时间开销、存储开销、可解释性等方面的因素进行评估并做出选择
- ✓ 假设测试集是从样本真实分布中独立采样获得 (i.i.d), 将测试集上的“测试误差”作为泛化误差的近似, 所以测试集要和训练集中的样本尽量互斥。

狭义的：

- 样例集：训练数据
 - 划分为两个互斥（不相交，并集为全集）集合：数据分布的一致性
 - ✓ 训练集S：训练
 - ✓ 测试集T：评估
 - ✓ 比例：2:1 到 4:1

广义的：

- 样例集：训练数据
 - 划分为两个互斥集合：数据分布的一致性
 - ✓ 训练集-S：训练
 - ✓ 验证集-T：评估
- 实际数据：测试数据-应用

通常将包含 m 个样本的数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$
拆分成训练集 S 和测试集 T ：（一个样本包含 x 和 y ）

关键：怎么获得“测试集” (test set) ？

测试集应该与训练集“互斥”

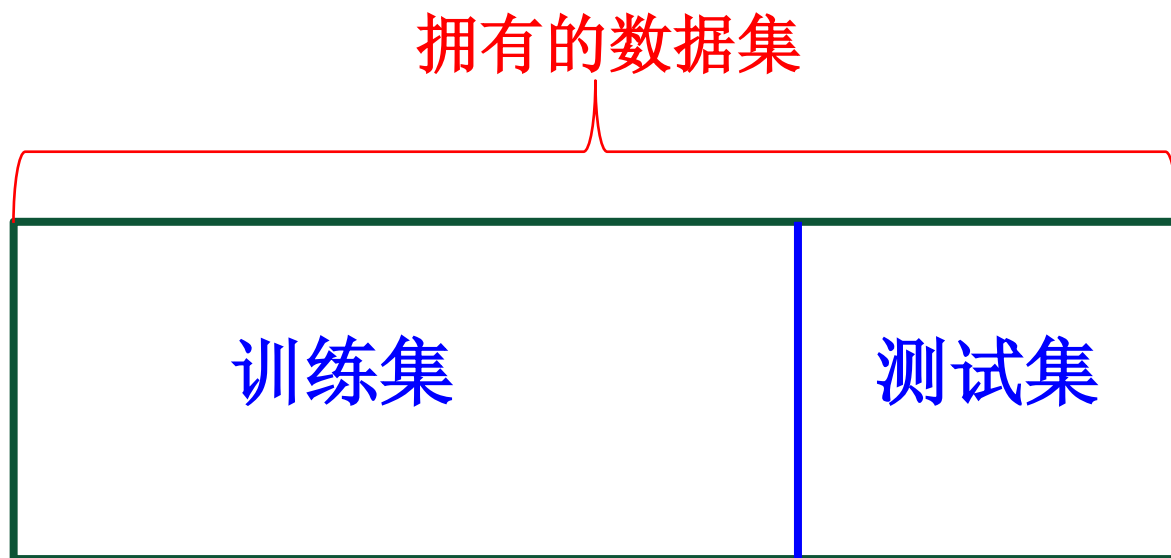
常见方法：

- 留出法 (hold-out)
- 交叉验证法 (cross validation)
- 自助法 (bootstrap)

□ 评估方法—— (1) 留出法

□ 留出法:

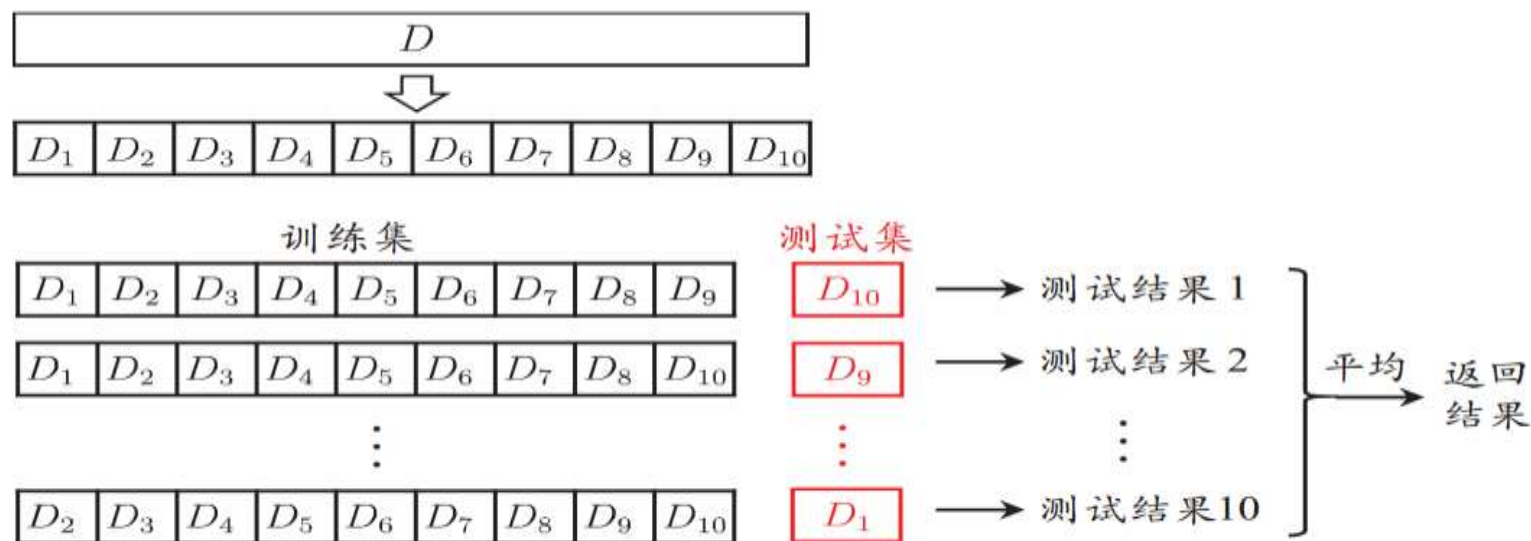
- ✓ 直接将数据集划分为两个互斥集合
- ✓ 训练/测试集划分要尽可能保持数据分布的一致性
- ✓ 一般若干次随机划分、重复实验取平均值
- ✓ 训练/测试样本比例: 2:1~4:1



□ 评估方法—— (2) 交叉验证法

□ 交叉验证法:

- ✓ 将数据集分层采样划分为 k 个大小相似的互斥子集，每次用 $k-1$ 个子集的并集作为训练集，余下的子集作为测试集，最终返回 k 个测试结果的均值， k 最常用的取值是10.



10 折交叉验证示意图

可重复性?

- 与留出法类似，将数据集D划分为k个子集同样存在多种划分方式，为了减小因样本划分不同而引入的差别，k折交叉验证通常随机使用不同的划分重复p次，最终的评估结果是这p次k折交叉验证结果的均值，例如常见的“10次10折交叉验证”

- 假设数据集D包含m个样本，若令 $k = m$ ，则得到留一法：
 - ✓ 不受随机样本划分方式的影响
 - ✓ 结果往往比较准确
 - ✓ 当数据集比较大时，计算开销难以忍受

□ 自助法

- ✓ 以自助采样法为基础，对数据集 D 有放回采样 m 次得到训练集 D' ，用 $D \setminus D'$ 做测试集。
- ✓ m 次有放回重复抽样
- ✓ 样本在 m 次采样中始终不被采到的概率

$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m \mapsto \frac{1}{e} \approx 0.368$$

- ✓ 实际模型与预期模型都使用 m 个训练样本
- ✓ 约有1/3的样本没在训练集中出现
- ✓ 从初始数据集中产生多个不同的训练集，对集成学习有很大的好处
- ✓ 自助法在数据集较小、难以有效划分训练/测试集时很有用；
- ✓ 由于改变了数据集分布可能引入估计偏差，在数据量足够时，留出法和交叉验证法更常用。

□ 调参与最终模型

□ 参数

- ✓ 算法参数-超参数：几个=人工调节
- ✓ 模型参数：很多=学习算法训练确定

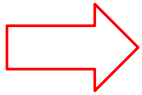
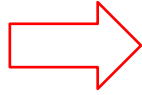
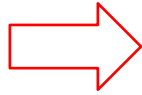
□ 最终模型

- ✓ 验证集：用于模型评估与选择中用于评测的数据
- ✓ 模型选择和调参：基于验证集上的性能
- ✓ 最终模型训练：@全部数据

□ 三种不同的集合的直观解释：

- ✓ 训练集(train):上课学到的知识
- ✓ 验证集(validation):课后习题，强化和纠正上课学到的知识（目的是调整超参数，找到最佳超参数）
- 测试集(test):考试，测试举一反三的能力（泛化能力）

三个关键问题：

- 如何获得测试结果？  评估方法
- 如何评估性能优劣？  性能度量
- 如何判断实质差别？  比较检验

第2章 模型评估与选择

2.1 经验误差与过拟合

2.2 评估方法



2.3 性能度量

2.4 比较检验

2.5 偏差与方差

评价指标



□ 性能度量—— (1) 回归任务

■ 均方误差MSE

□ 性能度量是衡量模型泛化能力的评价标准，反映了任务需求；使用不同的性能指标往往会**导致不同的评判结果**

□ 在**预测**任务中，给定样例集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$

评估学习器的性能 f 也即把预测结果 $f(x)$ 和真实标记比较.

□ **回归任务**最常用的性能度量是“**均方**误差 (Mean Square Error)”：

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m (f(x_i) - y_i)^2$$

均 (mean) 方(square)

□ 性能度量—— (2) 分类任务

■ 错误率和精度

□ 对于分类任务, 错误率和精度是最常用的两种性能度量:

□ 错误率: 分错样本占样本总数的比例

□ 精度: 分对样本占样本总数的比率

分类错误率

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) \neq y_i)$$

精 度

$$\begin{aligned} \text{acc}(f; D) &= \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) = y_i) \\ &= 1 - E(f; D) . \end{aligned}$$

□ 性能度量—— (2) 分类任务

■ 查准率、查全率和F1

- 查准率（精确率）——产全率（召回率）：信息检索、Web搜索等场景中经常需要衡量正例被预测出来的比率或者预测出来的正例中正确的比率，此时精确率(Precision)和召回率(Recall)比错误率和精度更适合。
- 统计真实标记和预测结果的组合可以得到“混淆矩阵”

分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

- ✓ TP (True Positive),
- ✓ FP (False Positive),
- ✓ TN(True Negative),
- ✓ FN(False Negative)

查准率/精确率

$$P = \frac{TP}{TP + FP}$$

查全率/召回率

$$R = \frac{TP}{TP + FN}$$

■ 查准率、查全率和 $F1$

混淆矩阵 Confusion Matrix

分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

- ✓ True Positive(真正, TP): 将正类预测为正类数.
- ✓ True Negative(真负, TN): 将负类预测为负类数.
- ✓ False Positive(假正, FP): 将负类预测为正类数 → → 误报 (Type I error).
- ✓ False Negative(假负, FN): 将正类预测为负类数 → → 漏报 (Type II error).

■ 查准率、查全率和F1

查准率/精确率 Precision

□ **查准率/精确率**是针对**预测结果**而言的，它表示的是预测为正的样本中有多少是真正的正样本。预测为正就有两种可能：一种就是把正类预测为正类(TP)，另一种就是把负类预测为正类(FP)，也就是

$$P = \frac{TP}{TP + FP}$$

查全率/召回率 Recall

□ **召回率**是针对原来的样本而言的，它表示样本中的正例有多少被预测正确了。有两种可能：一种是把原来的正类预测成正类(TP)，另一种就是把原来的正类预测为负类(FN)。

$$R = \frac{TP}{TP + FN}$$

■ 查准率、查全率和F1

案例：保安的能力评估

□ 保安对进出公司的人员进行盘查，识别其中的小偷。保安有可能犯两类错误：

- ✓ 1. 把好人误认为小偷，称为假小偷（与之相反称为真小偷）。
- ✓ 2. 把小偷误认为好人，称为假好人（与之相反称为真好人）。

□ 保安优秀与否可以按照如下的两个标准来评判：

- ✓ 1. 抓住的小偷中真小偷的比例尽量的高。
- ✓ 2. 宁可错抓好人也不放过小偷。

□ 标准1为**精准率** (Precision)

□ 标准2为**召回率** (Recall)

$$Precision = \frac{\text{抓住的真小偷数量}}{\text{抓住的真小偷数量} + \text{抓住的假小偷数量}}$$

$$Recall = \frac{\text{抓住的真小偷数量}}{\text{抓住的真小偷数量} + \text{放走的假好人数量}}$$

□ 性能度量—— (2) 分类任务

■ 查准率、查全率和F1

案例：保安的能力评估

□ 假设某公司每天进出100人，其中80个好人，20个小偷，某保安A的表现如下：

	实际上是小偷	实际上是好人	合计
抓住的小偷	5	1	6
放走的好人	15	79	94
合计	20	80	100

□ 保安共计抓住6个小偷，其中5人是真小偷，另外1人是假小偷（好人被误抓），依据公式可知，该保安的Precision=5/(5+1)=5/6≈88.3%。

□ 看上去这名保安抓人的准确性挺强，但他是一名好保安吗？

保安A

□ 性能度量—— (2) 分类任务

■ 查准率、查全率和F1

案例：保安的能力评估

□ 来看看召回率的计算。Recall=5/(5+15)=1/4=25%。放走的假好人就是被保安误认为好人的小偷。所以Recall的分母总是等于小偷的总数（而Precision的分母是抓住的人的总数）。从召回率可以看出，这名保安放走了3/4的小偷，这实在不能算优秀的保安。

□ 某保安B的表现如下：

	实际上是小偷	实际上是好人	合计
抓住的小偷	20	20	40
放走的好人	0	60	60
合计	20	80	100

□ 根据上述公式计算，Precision=50%，Recall=100%。虽然精度不高，误抓了不少好人，但这名保安抓住了全部小偷。

保安
B

性能度量—— (2) 分类任务

查准率、查全率和F1

案例：保安的能力评估

理想的保安：

	实际上是小偷	实际上是好人	合计
抓住的小偷	20	0	20
放走的好人	0	80	80
合计	20	80	100

Precision=Recall=100%。

将Precision和Recall放到一个公式中，就构成了F1 Score，是精度和召回率的调和均值，两者都高则F1高，其公式如下：(F1一定<1)

$$F_1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2TP}{2TP + FN + FP}$$

■ 查准率、查全率和 $F1$

指标分析

- 准确率是评估捕获的成果中目标成果所占得比例；召回率就是从关注领域中，召回目标类别的比例；而 $F1$ 值，则是综合这二者指标的评估指标，用于综合反映整体的指标。
- 当然希望检索结果Precision越高越好，同时Recall也越高越好，**但事实上这两者在某些情况下有矛盾的**。比如极端情况下，只搜索出了一个结果，且是准确的，那么Precision就是100%，但是Recall就很低；而如果把所有结果都返回，那么比如Recall是100%，但是Precision就会很低。**因此在不同的场合中需要自己判断希望Precision比较高或是Recall比较高。**
- 可以绘制Precision-Recall曲线来帮助分析。

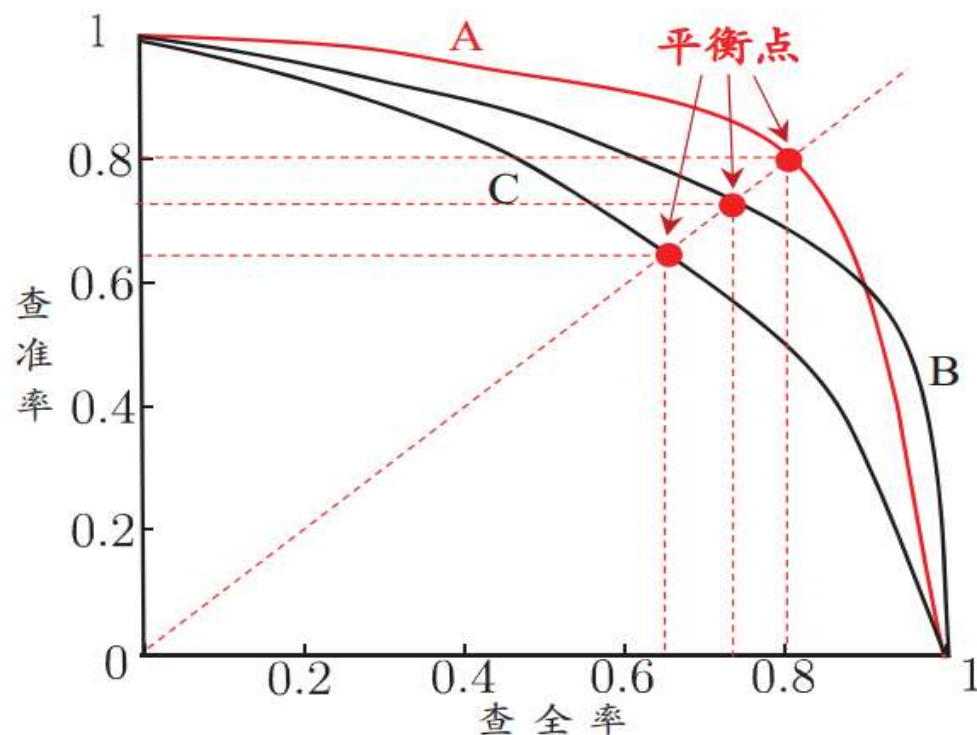
□ 性能度量—— (2) 分类任务

■ 性能度量——平衡P-R曲线

- 根据学习器的预测结果按正例可能性大小对样例进行排序，并逐个把样本作为正例进行预测，则可以得到查准率-查全率曲线，简称

“P-R曲线”

- 平衡点是曲线上“查准率=查全率”时的取值，可用来用于度量P-R曲线有交叉的分类器性能高低

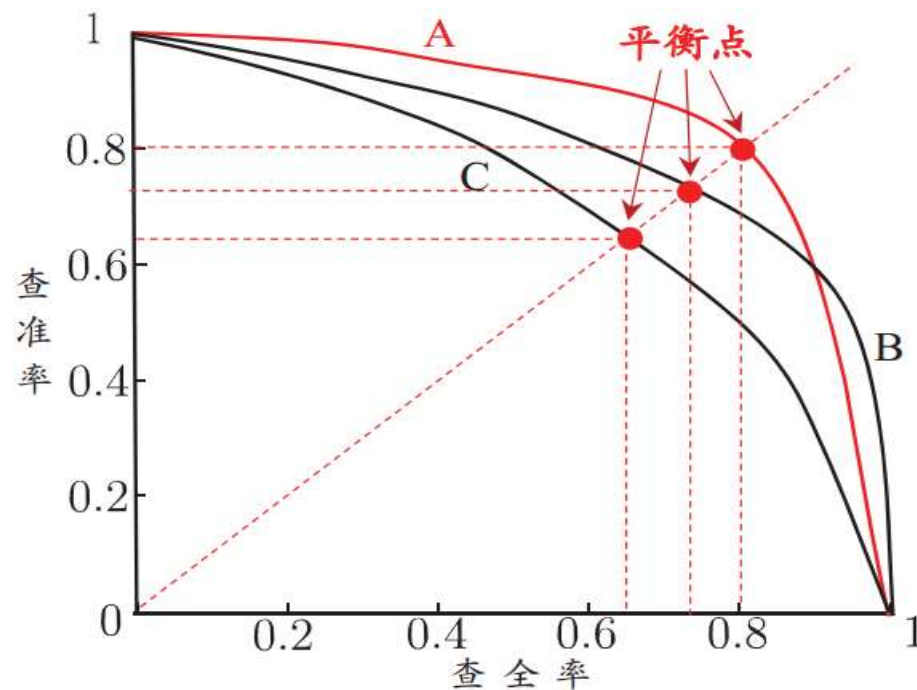


P-R曲线与平衡点示意图

■ 性能度量——平衡P-R曲线

■ “P-R曲线” 绘制步骤:

- ✓ 样例排序
- ✓ 迭代 {逐一分界-计算P/R-绘散点}
- ✓ 联折线



P-R曲线与平衡点示意图

■ 性能度量——平均与侧重P-R

- ✓ 比P-R曲线平衡点更常用的是F1度量:

$$F1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times TP}{\text{样例总数} + TP - TN}$$

- ✓ 比F1更一般的形式 F_β

$$F_\beta = \frac{(1 + \beta^2) \times P \times R}{(\beta^2 \times P) + R}$$

$\beta = 1$: 标准F1

$\beta > 1$: 偏重查全率(逃犯信息检索)

$\beta < 1$: 偏重查准率(商品推荐系统)

■ ROC与AUC

- (1) 真正类率 (True Positive Rate , **TPR**) 【灵敏度 (sensitivity)】 :

$TPR = TP / (TP + FN)$, 即正样本预测结果数/正样本实际数

- (2) 假负类率 (False Negative Rate , **FNR**) : $FNR = FN / (TP + FN)$, 即被预测为负的正样本结果数/正样本实际数

- (3) 假正类率 (False Positive Rate , **FPR**) : $FPR = FP / (FP + TN)$, 即被预测为正的负样本结果数/负样本实际数

- (4) 真负类率 (True Negative Rate , **TNR**) 【特指度 (specificity)】 : $TNR = TN / (FP + TN)$, 即负样本预测结果数/负样本实际数

■ ROC曲线

- ROC曲线指受试者工作特征曲线 (receiver operating characteristic curve),
- 反映敏感性和特异性连续变量的综合指标, 是用构图法揭示敏感性和特异性的相互关系。

ROC曲线的横轴和纵轴:

■ 纵轴: **TPR (真正例率)** = $TP / (TP + FN)$

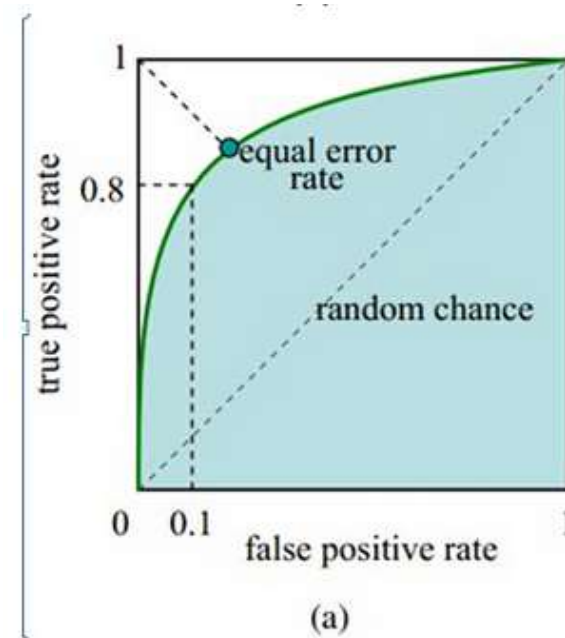
✓ 通俗理解: “敏感度”, 也就是在所有真实的“正类” (比如病人) 中, 模型成功揪出来了多少。

✓ 愿望: 越高越好。

■ 横轴: **FPR (假正例率)** = $FP / (FP + TN)$

✓ 通俗理解: “特异性”, 在所有真实的“负类” (比如健康人) 中, 模型误伤了多少。

✓ 愿望: 越低越好。



■ ROC曲线——如何画曲线？理解“阈值”

这是最核心的一步。模型通常输出一个概率（比如：患病概率 0.7），我们需要设定一个阈值（比如 0.5），大于它就算患病。

◆ **场景A（阈值设得很低，比如 0.1）**：只要有一点像病人，我就判为有病。

✓ 结果：TPR 会很高（因为几乎所有病人都被抓住了），但 FPR 也会很高（很多健康人被误伤）。表现在图上，就是右上角的点。

◆ **场景B（阈值设得很高，比如 0.9）**：除非非常确定，否则我不判为有病。

✓ 结果：FPR 会很低（很少误伤健康人），但 TPR 也会很低（漏掉了好多病人）。表现在图上，就是左下角的点。

◆ **场景C（把所有可能的阈值都试一遍）**：从 1 慢慢降到 0，我们会得到无数个 (FPR, TPR) 的点。把这些点连起来，就是 **ROC曲线**。

核心结论： ROC曲线本质上就是随着分类器阈值的改变，所有分类结果的“快照”连成的轨迹。

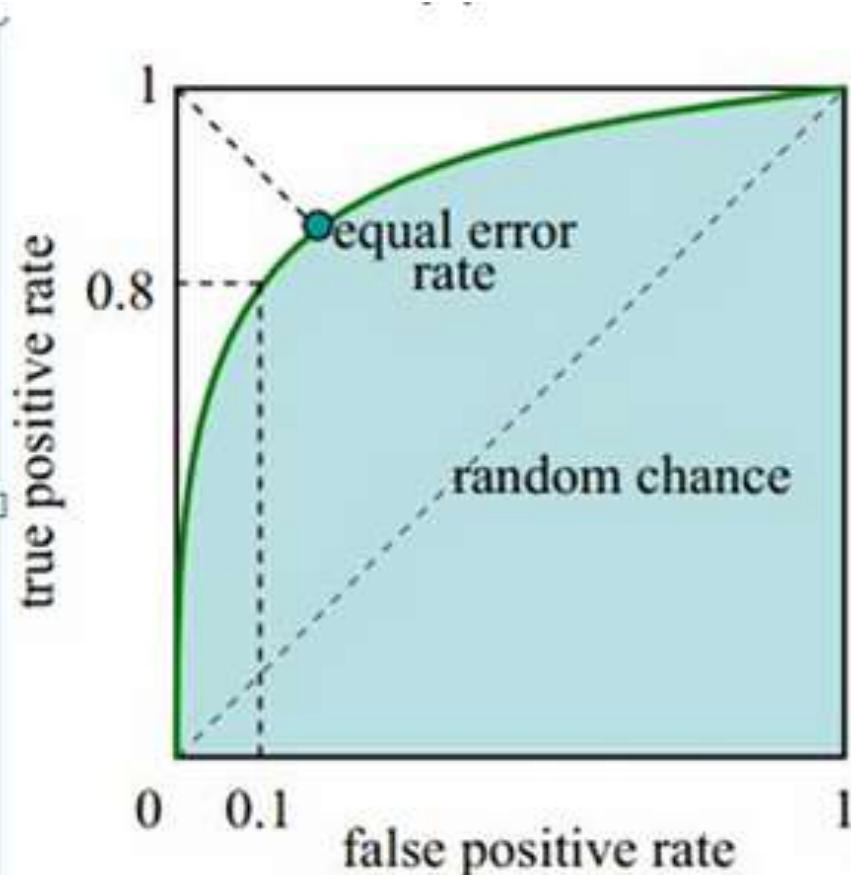
■ ROC曲线

- 比如在逻辑回归算法里面，设一个阈值，大于这个值的为正类，小于这个值为负类。如果减小这个阈值，那么更多的样本会被识别为正类。这会提高正类的识别率，但同时也会使得更多的负类被错误识别为正类。
- 假设采用逻辑回归分类器，其给出针对每个实例为正类的概率，那么通过设定一个阈值如0.6，概率大于等于0.6的为正类，小于0.6的为负类。对应的就可以算出一组(FPR, TPR)，在平面中得到对应坐标点。
- 随着阈值的逐渐减小，越来越多的实例被划分为正类，但是这些正类中同样也掺杂着真正的负实例，即TPR和FPR会同时增大。阈值最大时，对应坐标点为(0, 0)，阈值最小时，对应坐标点(1, 1)。

□ 性能度量—— (2) 分类任务

■ ROC曲线——如何看懂ROC曲线？

□ 如下面这幅图，(a) 图中实线为ROC曲线，线上每个点对应一个阈值。

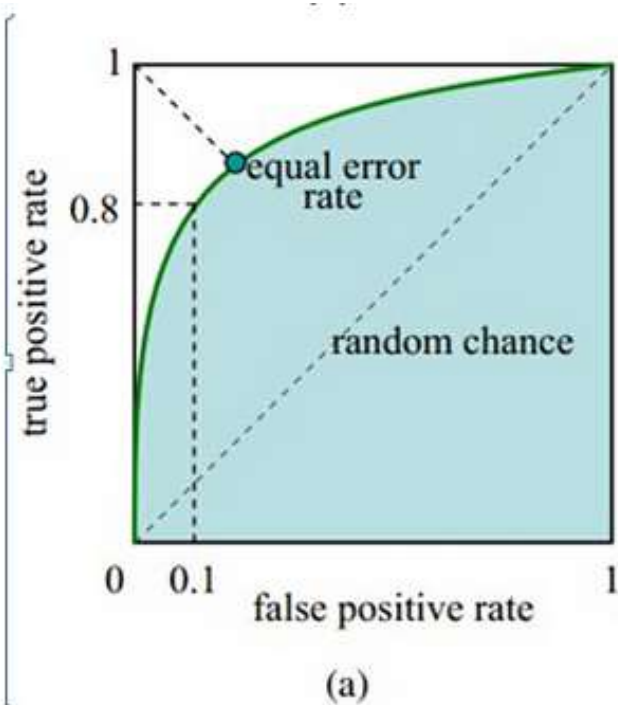


(a)

- ✓ 横轴FPR: $1 - \text{TNR}$, $1 - \text{Specificity}$, FPR越大, 预测正类中实际负类越多。
- ✓ 纵轴TPR: Sensitivity(正类覆盖率), TPR越大, 预测正类中实际正类越多。
- ✓ 理想目标: $\text{TPR} = 1$, $\text{FPR} = 0$, 即图中(0,1)点, 故ROC曲线越靠拢(0,1)点, 越偏离45度对角线越好, Sensitivity、Specificity越大效果越好。

ROC曲线

考虑ROC曲线图中的四个点和一条线。



- ✓ 第一个点， $(0,1)$ ，即 $FPR=0, TPR=1$ ，这意味着 FN (false negative) $=0$ ，并且 FP (false positive) $=0$ 。这是一个完美的分类器，它将所有的样本都正确分类。虽然现实中几乎不可能，但曲线越靠近这个点，模型性能越好。
- ✓ 第二个点， $(1,0)$ ，即 $FPR=1, TPR=0$ ，类似地分析可以发现这是一个最糟糕的分类器，因为它成功避开了所有的正确答案。
- ✓ 第三个点， $(0,0)$ ，即 $FPR=TPR=0$ ，即 FP (false positive) $=TP$ (true positive) $=0$ ，可以发现该分类器预测所有的样本都为负样本 (negative)。
- ✓ 第四个点 $(1,1)$ ，分类器实际上预测所有的样本都为正样本。经过以上的分析，可以断言，ROC曲线越接近左上角，该分类器的性能越好。

ROC曲线图中的虚线 $y=x$ 上的点。这条对角线上的点其实表示的是一个采用随机猜测策略的分类器的结果，例如 $(0.5,0.5)$ ，表示该分类器随机对于一半的样本猜测其为正样本，另外一半的样本为负样本。

■ AUC指标 (ROC曲线下面积)

- ✓ AUC (Area Under Curve) 被定义为ROC曲线下的面积, 显然这个面积数值不会大于1。又由于ROC曲线一般都处于 $y=x$ 这条直线的上方, 所以AUC的取值范围在0.5和1之间。
- ✓ 使用AUC值作为评价标准是因为很多时候ROC曲线并不能清晰的说明哪个分类器的效果更好, 而AUC作为数值可以直观的评价分类器的好坏, 值越大越好。
- ✓ 首先AUC值是一个概率值, 当随机挑选一个正样本以及一个负样本, 当前的分类算法根据计算得到的Score值将这个正样本排在负样本前面的概率就是AUC值 (比如100%和50%)。当然, AUC值越大, 当前的分类算法越有可能将正样本排在负样本前面, 即能够更好的分类。

■ AUC指标

□ AUC值为ROC曲线所覆盖的区域面积，显然AUC越大，分类器分类效果越好。

AUC = 1，是完美分类器，采用这个预测模型时，不管设定什么阈值都能得出完美预测。绝大多数预测的场合，不存在完美分类器。

0.5 < AUC < 1，优于随机猜测。这个分类器（模型）妥善设定阈值的话，能有预测价值。

AUC = 0.5，跟随机猜测一样（例：丢铜板），模型没有预测价值。

AUC < 0.5，比随机猜测还差；但只要总是反预测而行，就优于随机猜测。

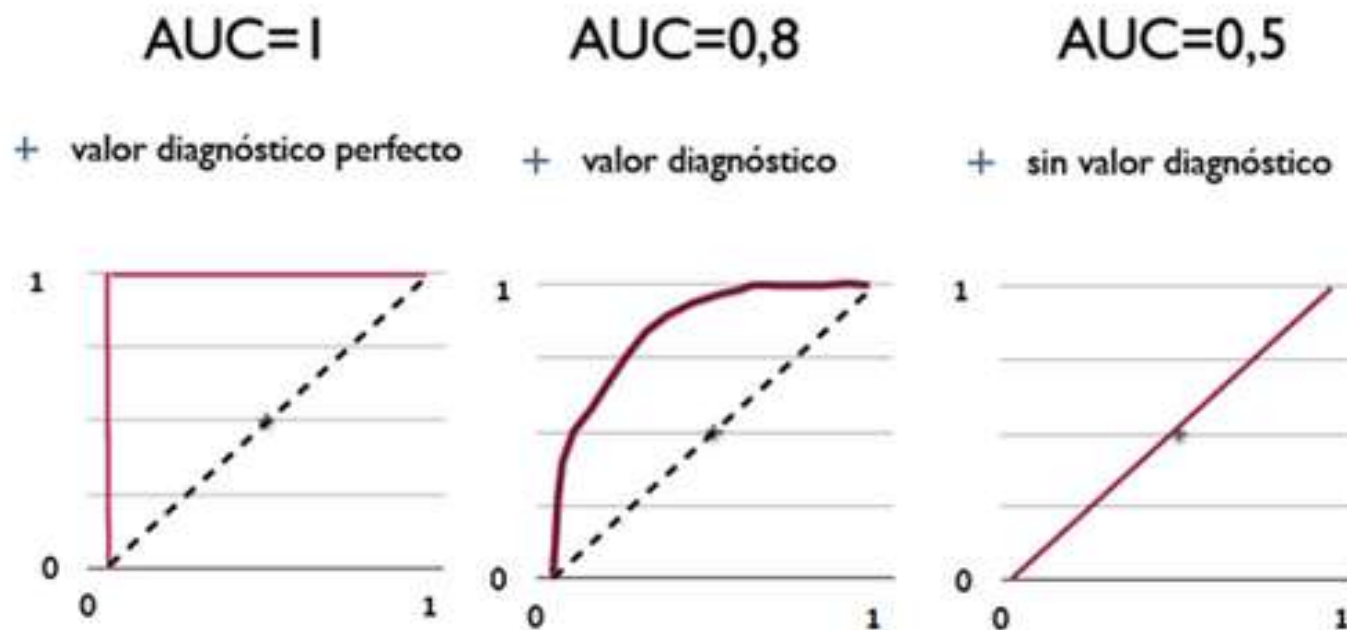
0.5是最没有意义的

以下示例三种AUC值（曲线下面积）：

□ 性能度量—— (2) 分类任务

■ AUC指标物理意义

□ 假设分类器的输出是样本属于正类的score（置信度），则AUC的物理意义为，任取一对（正、负）样本，正样本的score大于负样本的score的概率。



■ 为什么需要ROC和AUC

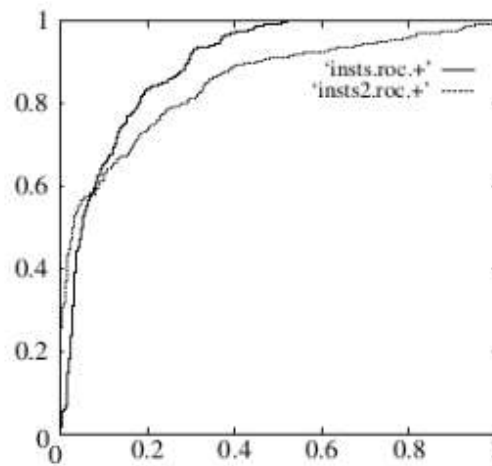
- 既然已经有这么多评价标准，为什么还要使用ROC和AUC呢？
- 因为ROC曲线有个很好的特性：当测试集中的正负样本的分布变化的时候，ROC曲线能够保持不变。
- 在实际的数据集中经常会出现类不平衡（class imbalance）现象，即负样本比正样本多很多（或者相反），而且测试数据中的正负样本的分布也可能随着时间变化。下图是ROC曲线和Precision-Recall曲线的对比：

■ 为什么需要ROC和AUC

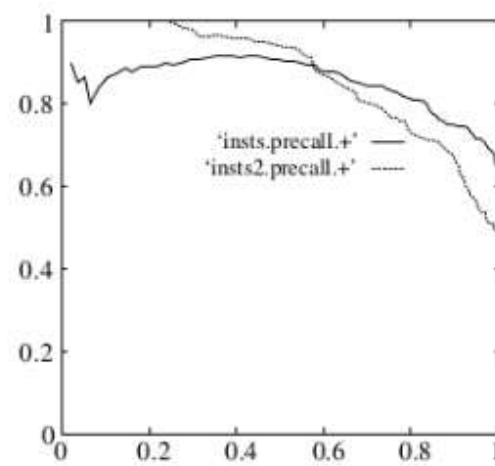
ROC曲线和Precision-Recall曲线的对比：

□ (a)和(c)为ROC曲线，(b)和(d)为 Precision-Recall曲线。

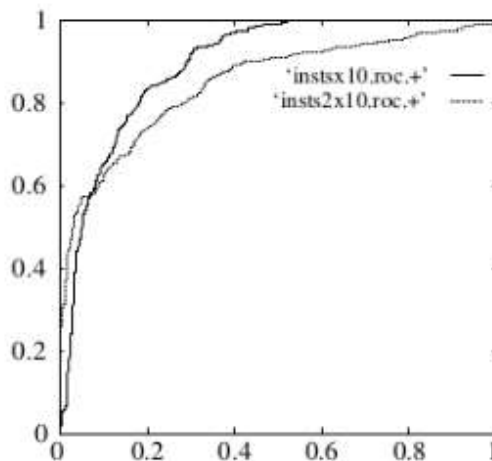
□ (a)和(b)展示的是分类其在原始测试集（正负样本分布平衡）的结果，(c)和(d)是将测试集中负样本的数量增加到原来的10倍后，分类器的结果。可以明显的看出，ROC曲线基本保持原貌，而Precision-Recall曲线则变化较大



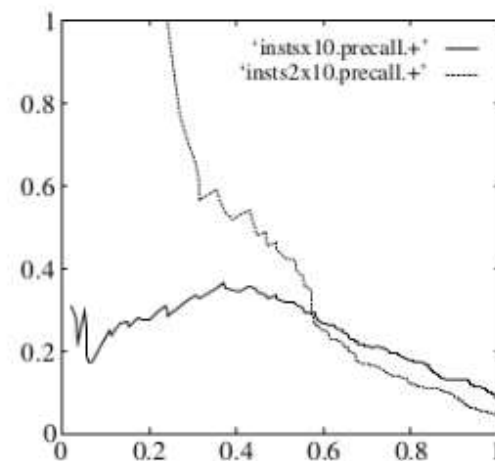
(a)



(b)



(c)



(d)

第2章 模型评估与选择

2.1 经验误差与过拟合

2.2 评估方法

2.3 性能度量

➡ 2.4 比较检验

2.5 偏差与方差

■ 比较检验

- 比较检验就是一套统计学方法，目的是帮我们判断：模型之间表现出的性能差异，到底是真的因为模型本身有优劣，还是仅仅因为测试数据的随机选择导致的“运气好”。
- 一个通俗的比喻：两位射击运动员
假设有两位射击运动员（A和B），每人射三枪。
 - ✓ 运动员A的成绩：10环、9环、8环。
 - ✓ 运动员B的成绩：9环、8环、7环。

只看平均环数：A平均9环，B平均8环，A比B强。
- 但统计学家的疑问是：如果让A再射三枪，他还能保证每次都赢B吗？会不会A这次只是运气好？B只是今天感冒了状态不佳？
- 比较检验，就是通过一套数学方法，来计算“A比B强”这个结论到底有多可靠。

■ 比较检验—为什么不能只看指标直接比？

在机器学习中，这个“运气”主要来自两个方面：

1. **测试集的随机性**：我们用来评估模型的数据，只是全部可能数据中的一个样本。换一批测试数据，准确率可能会波动。
2. **算法本身的随机性**：很多算法（如神经网络的随机初始化、随机森林的抽样）本身就带有随机性，多次运行结果也会不同。

例子：

- ✓ 模型A准确率95%，模型B准确率94%。
- ✓ 如果测试集只有100条数据，这1%的差异其实只差了1条数据。很可能只是因为这1条数据恰好对B不利。这个差异在统计上可能并不显著。

■ 比较检验

□ 关于性能比较：

- ✓ 测试性能并不等于泛化性能
- ✓ 测试性能随着测试集的变化而变化
- ✓ 很多机器学习算法本身有一定的随机性

直接选取相应评估方法在相应度量下比大小的方法不可取！

- **假设检验**为学习器性能比较提供了重要依据，基于其结果我们可以推断出若在测试集上观察到学习器A比B好，则A的泛化性能是否在统计意义上优于B，**以及这个结论的把握有多大。**

■ 主要的比较检验方法

1. 单个模型 vs 固定值（检验模型本身是否有效）

- ✓ 场景：训练了一个分类模型，准确率达到 60%。而瞎猜的准确率是 50%。这60%真的比瞎猜好吗？
- ✓ 方法：常用**假设检验**。它会计算在瞎猜的前提下（抛硬币），得到60%或更高准确率的概率有多大。如果这个概率极低（比如小于0.05），说明你的模型确实比瞎猜强。

2. 两个模型之间的比较（最常用）。训练了模型A和模型B，想比较谁更好。

- ✓ 如果有多组独立测试集（比如K折交叉验证的结果）：
 - **配对t检验**：把模型A在每一折上的得分和模型B在每一折上的得分当成一对一对的数据，看看这对数据的差值是否显著偏离0。如果有显著偏离，说明一个比另一个好。
- ✓ 如果只有一个测试集，且想对比两个模型在每个样本上的表现（比如A预测对了，B预测错了）：
 - **McNemar检验**：这是一种专门针对二分类问题，比较两个模型在同一个测试集上预测结果差异的检验。它关注的是两个模型预测结果不一致的那些样本（即A对B错，和A错B对的样本数量），看这种差异是否是偶然。

■ 主要的比较检验方法

3. 多个模型之间的比较（排序与选优）

• **场景：**同时训练了A、B、C三个模型（甚至更多，比如深度学习中的各种变体），想选出最好的，或者想给它们排个序。

✓ **Friedman检验：**这相当于模型比较界的“ANOVA 方差分析”。它先对每个数据集上各个模型的性能进行排名（第一名得1分，第二名得2分...），然后计算所有数据集上的平均排名。

Friedman检验就是看这些平均排名的差异是否显著。

✓ **Nemenyi后续检验：**如果Friedman检验发现模型之间存在显著差异，Nemenyi检验会进一步告诉我们，具体是哪两个模型之间有显著差异。通常配合**临界差图**来展示，图上那些连在一起的模型，说明它们的性能没有显著差别。

■ 假设检验

反复扔硬币应该符合二项分布

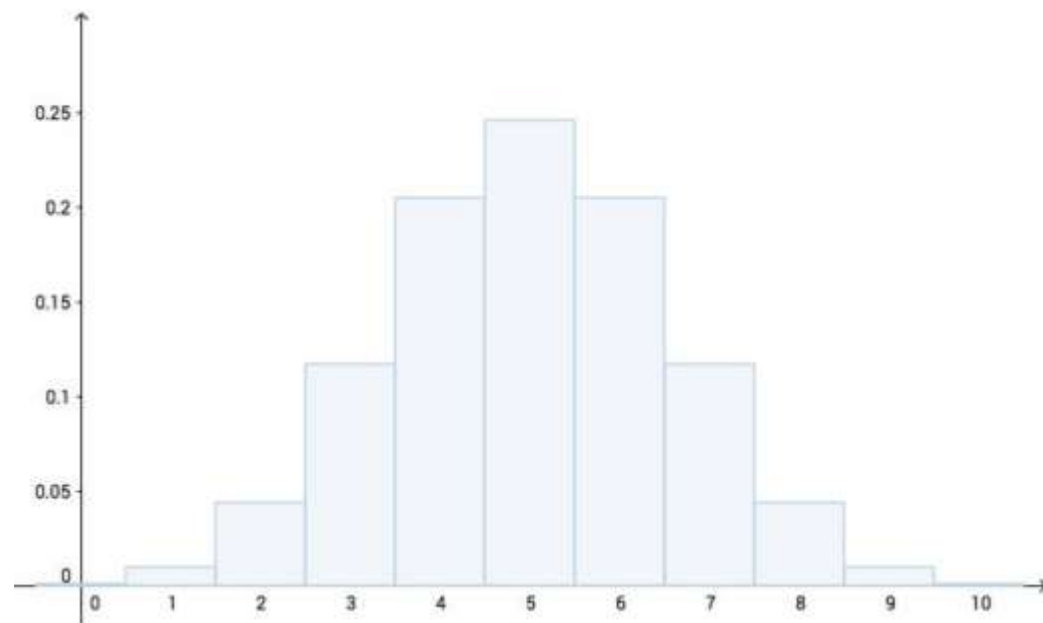
$$X \sim B(n, \mu)$$

其中, n 代表扔硬币的次数, μ 代表“花”朝上的概率。

在我们认为硬币是公平的前提下, 扔10次硬币应该符合以下分布:

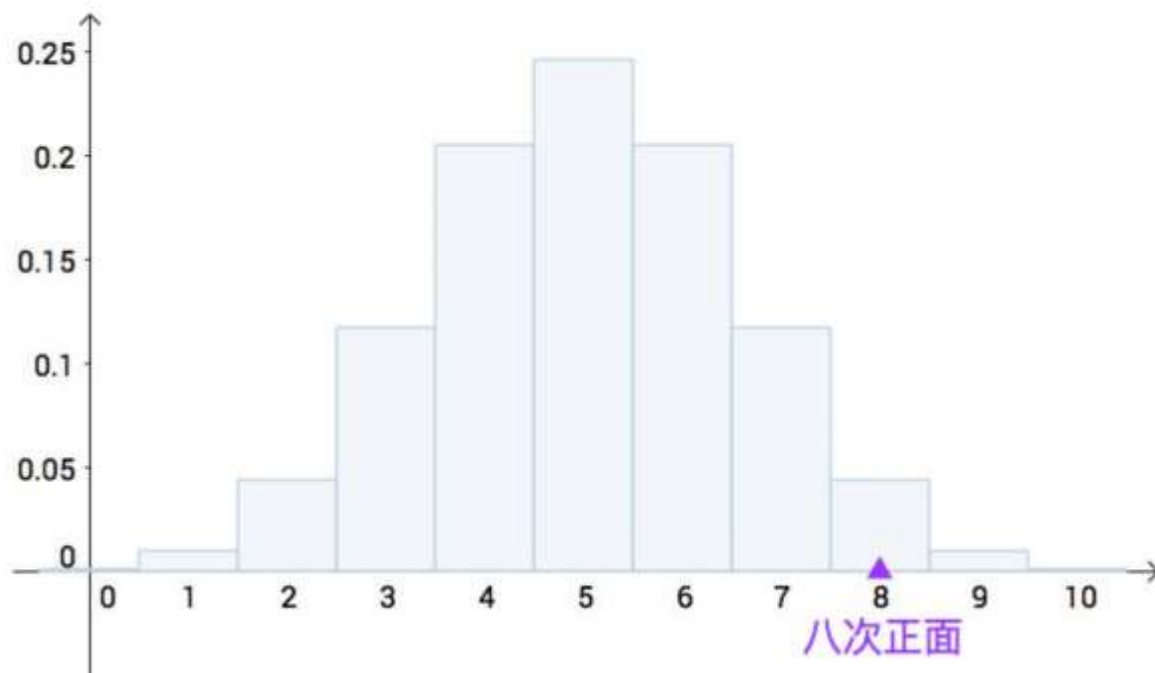
$$X \sim B(10, 0.5)$$

下图表示的就是, 假如硬币是公平的情况下的分布图:

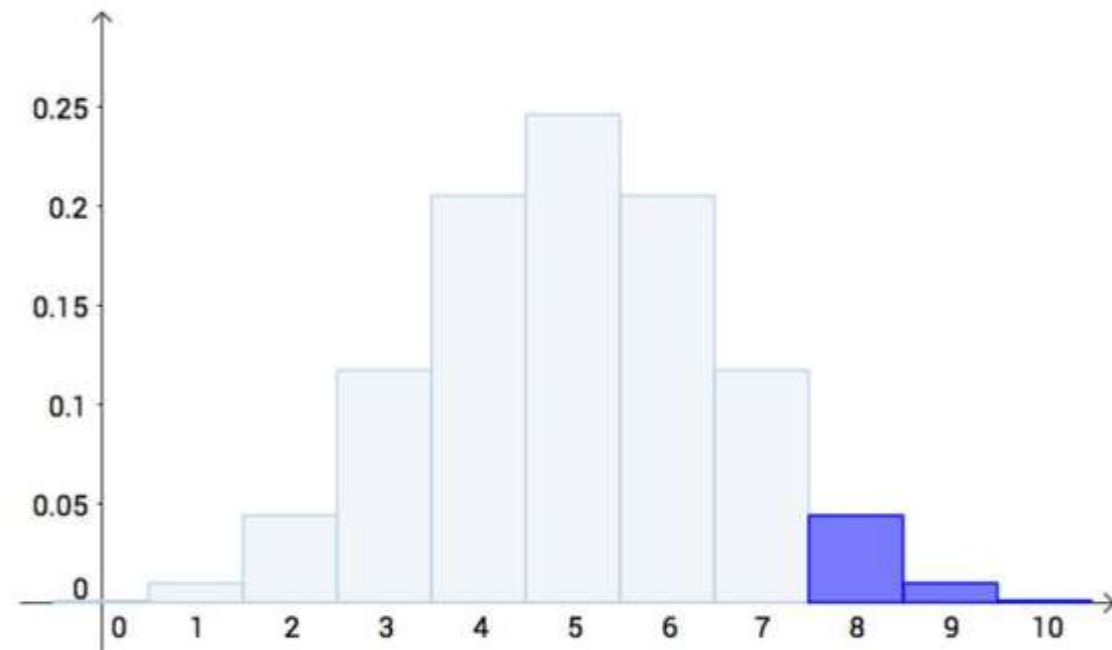


假设检验

扔了十次之后得到的结果是，有八次正面：



把八次正面的概率，与更极端的九次正面、十次正面的概率加起来：

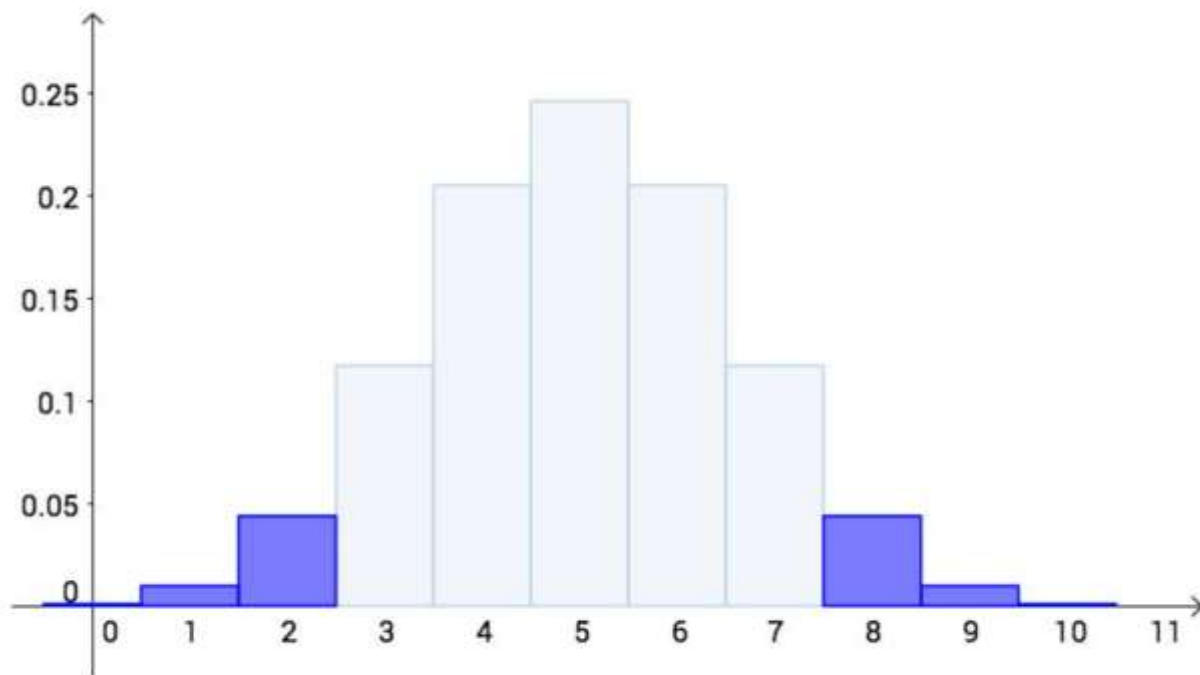


得到的就是（单侧P值）：

$$p\text{-value} = P(8 \leq X \leq 10) = 0.05$$

■ 假设检验

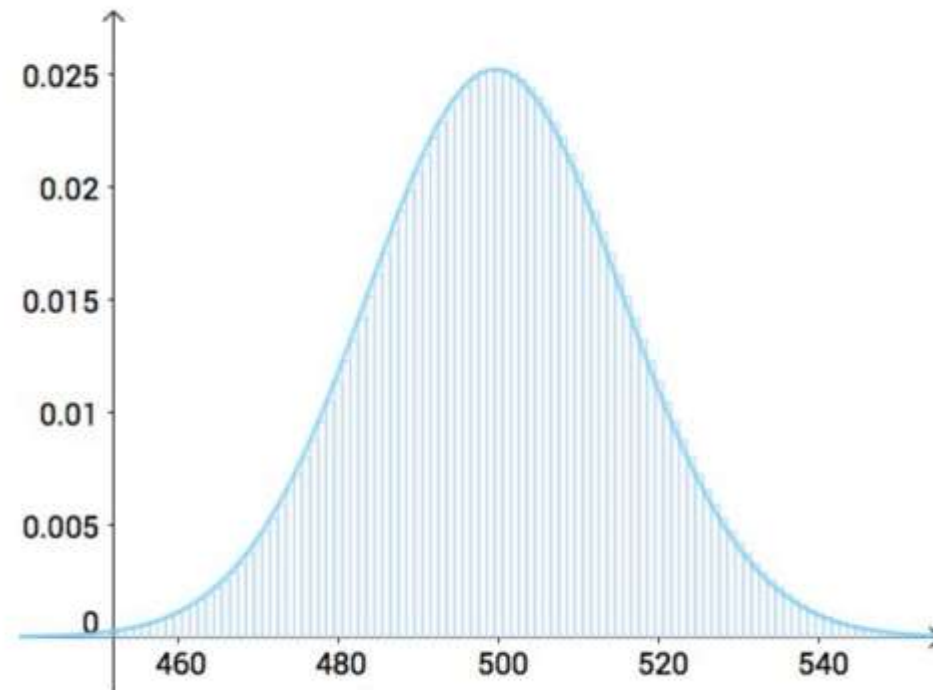
其实，出现两次正面、一次正面、零次正面的概率也是很极端的：



所以（双侧P值）：

$$\text{p-value} = P(0 \leq X \leq 2) + P(8 \leq X \leq 10) = 0.1$$

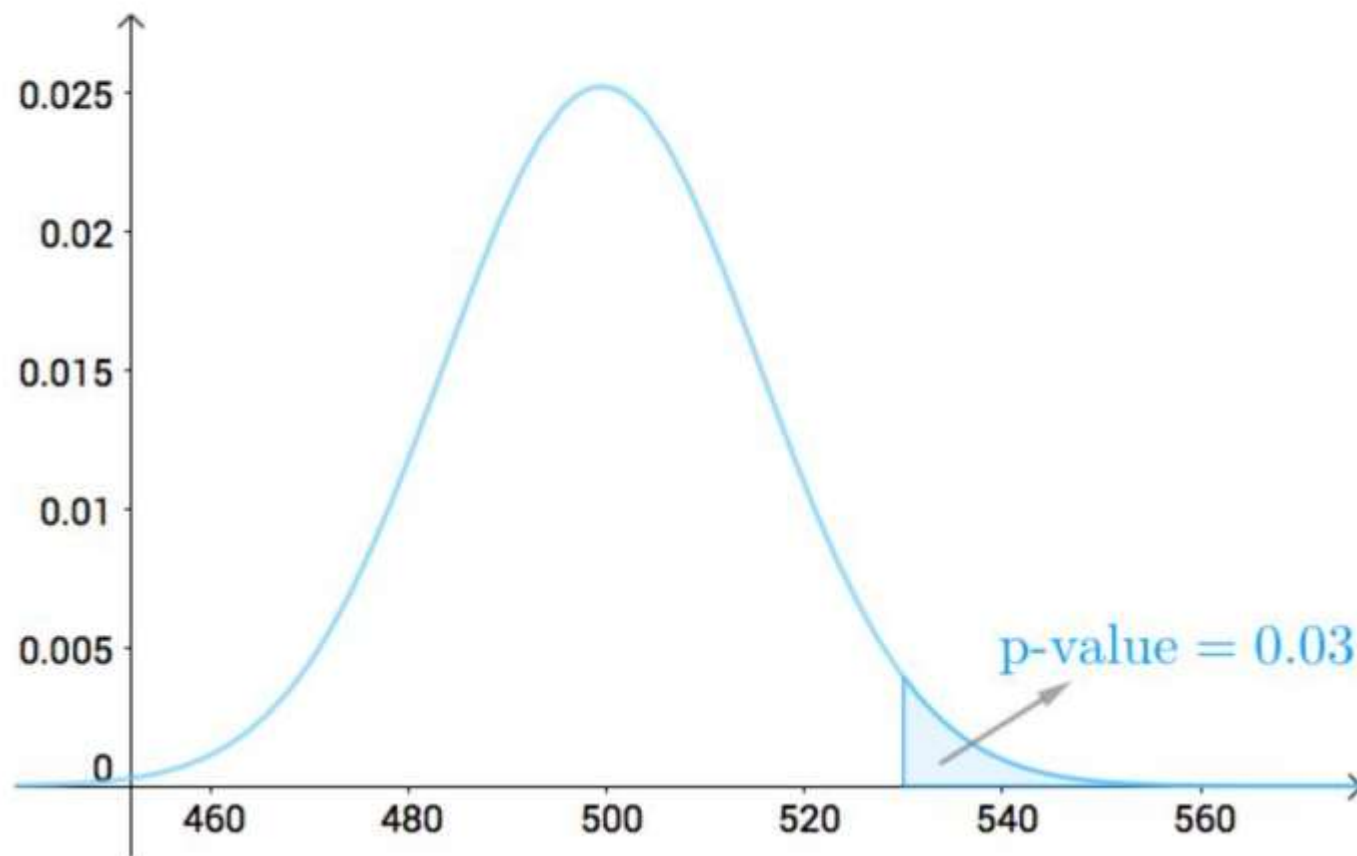
扔1000次硬币用二项分布来计算很麻烦，根据中心极限定理，我们知道，可以用正态分布来近似：



比如，我扔了1000次，得到了530次正面，用正态分布来计算就比较简单。

■ 假设检验

但是，对于正态分布，没有办法算单点的概率（连续分布单点概率为0），只能取一个区间来算极限，所以就取530、以及更极端的点组成的区间：



■ 假设检验

显著水平

总共扔10次硬币，那么是出现7次正面之后，可以认为“硬币是不公平的”，还是9次正面之后才能确认“硬币是不公平的”，这是一个较为主观的标准。

我们一般认为

$$\text{p-value} \leq 0.05$$

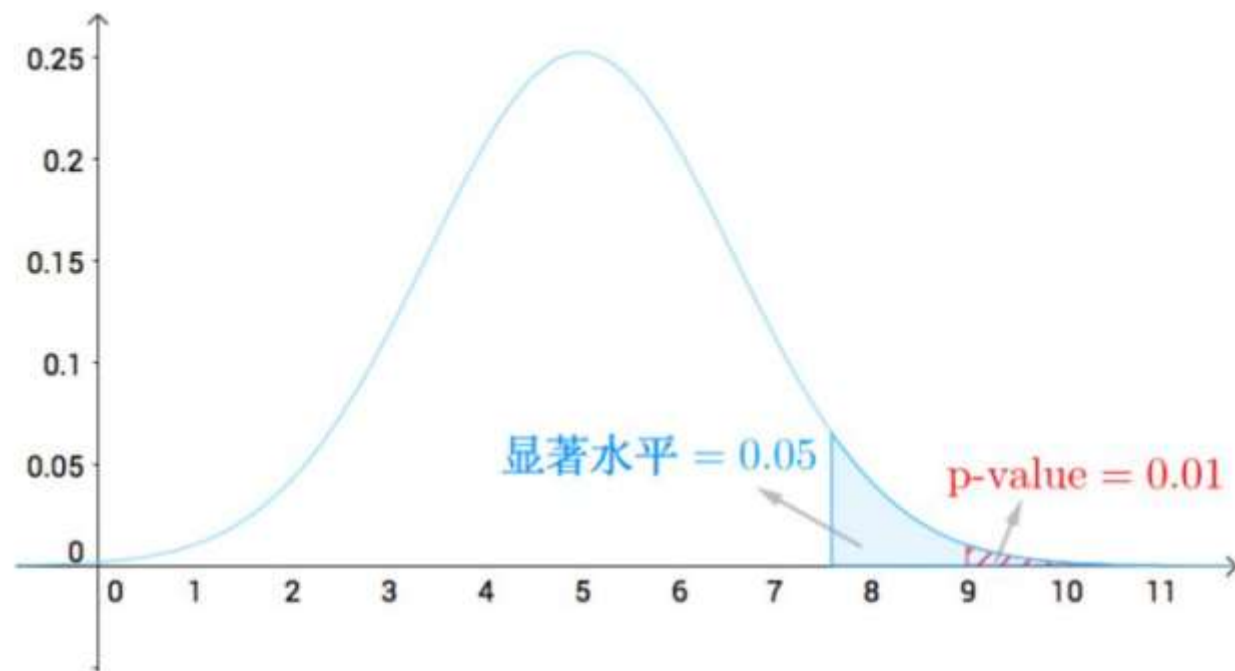
就可以认为假设是不正确的。

0.05这个标准就是**显著水平**，当然选择多少作为显著水平也是主观的。

比如，上面的扔硬币的例子，如果取单侧P值，那么根据我们的计算，如果扔10次出现9次正面：

$$\text{p-value} = P(9 \leq X \leq 10) = 0.01 \leq 0.05$$

■ 假设检验



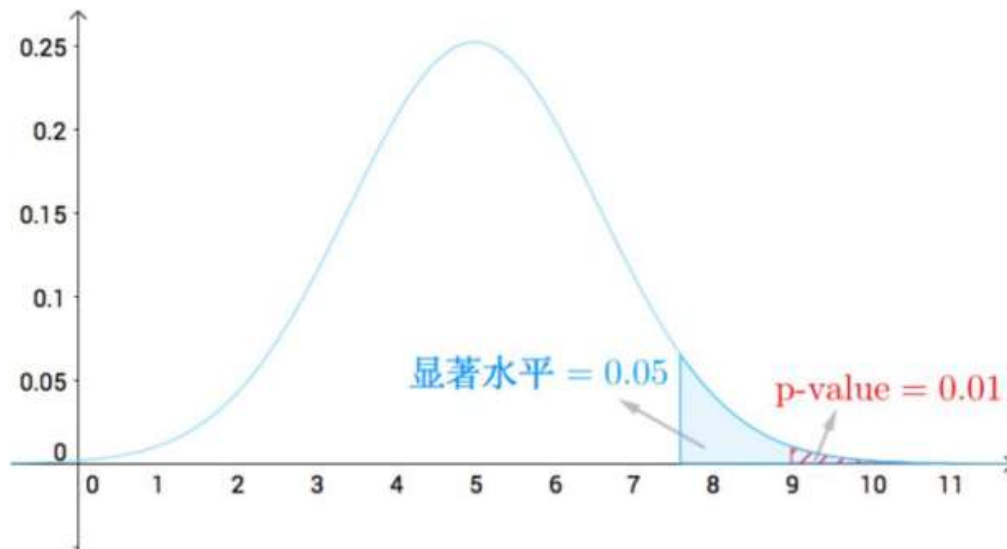
我们可以认为刚开始的假设错的很“显著”，也就是“硬币是不公平的”。

如果扔10次出现出现8次正面：

$$\text{p-value} = P(8 \leq X \leq 10) = 0.05 \leq 0.05$$

呃，这个和我们的显著水平是一样的啊，我们也可以拒绝假设，只是没有那么“显著”了。

■ 假设检验



我们可以认为刚开始的假设错的很“显著”，也就是“硬币是不公平的”。

如果扔10次出现出现8次正面：

$$p\text{-value} = P(8 \leq X \leq 10) = 0.05 \leq 0.05$$

呃，这个和我们的显著水平是一样的啊，我们也可以拒绝假设，只是没有那么“显著”了。

假设检验是统计学中用数据说话的标准语言。它通过计算“巧合的概率”（p值），帮助我们在随机性的世界中做出理性决策。

所以，下次你在论文中看到“ **$p < 0.05$** ，模型表现有显著提升”时，就可以理解作者想表达的是：这个提升不太可能是偶然撞大运撞出来的。

第2章 模型评估与选择

2.1 经验误差与过拟合

2.2 评估方法

2.3 性能度量

2.4 比较检验

➡ 2.5 偏差与方差

■ 性能解释

□ 通过实验可以估计学习算法的泛化性能，而“偏差-方差分解”可以用来帮助解释泛化性能。偏差-方差分解试图对学习算法期望的泛化错误率进行拆解。

对测试样本 x ，令 y_D 为 x 在数据集中的标记， y 为 x 的真实标记， $f(x; D)$ 为训练集 D 上学得模型 f 在 x 上的预测输出。以回归任务为例：学习算法的期望预期为：

$$\bar{f}(x) = \mathbb{E}_D[f(x; D)]$$

使用样本数目相同的不同训练集产生的方差为

$$\text{var}(x) = \mathbb{E}_D \left[(f(x; D) - \bar{f}(x))^2 \right]$$

噪声为

$$\varepsilon^2 = \mathbb{E}_D \left[(y_D - y)^2 \right]$$

■ 期望泛化误差分解

期望输出与真实标记的差别称为偏差，即 $bias^2(\mathbf{x}) = (\bar{f}(\mathbf{x}) - y)^2$
为便与讨论，假定噪声期望为0，也即 $\mathbb{E}_D[y_D - y] = 0$ ，对泛化误差分解

$$\begin{aligned} E(f; D) &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - y_D)^2 \right] \\ &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) - y_D)^2 \right] \\ &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + \mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y_D)^2 \right] \\ &\quad + \mathbb{E}_D \left[2(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))(\bar{f}(\mathbf{x}) - y_D) \right] \\ &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + \mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y_D)^2 \right] \end{aligned}$$

■ 偏差与方差

$$\begin{aligned} &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + \mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y + y - y_D)^2 \right] \\ &= \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + \mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y)^2 \right] + \mathbb{E}_D \left[(y - y_D)^2 \right] \\ &\quad + 2\mathbb{E}_D \left[(\bar{f}(\mathbf{x}) - y)(y - y_D) \right] \end{aligned}$$

又由假设中噪声期望为0，可得

$$E(f; D) = \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right] + (\bar{f}(\mathbf{x}) - y)^2 + \mathbb{E}_D \left[(y_D - y)^2 \right]$$

于是： $E(f; D) = bias^2(\mathbf{x}) + var(\mathbf{x}) + \varepsilon^2$

也即泛化误差可分解为偏差、方差与噪声之和。

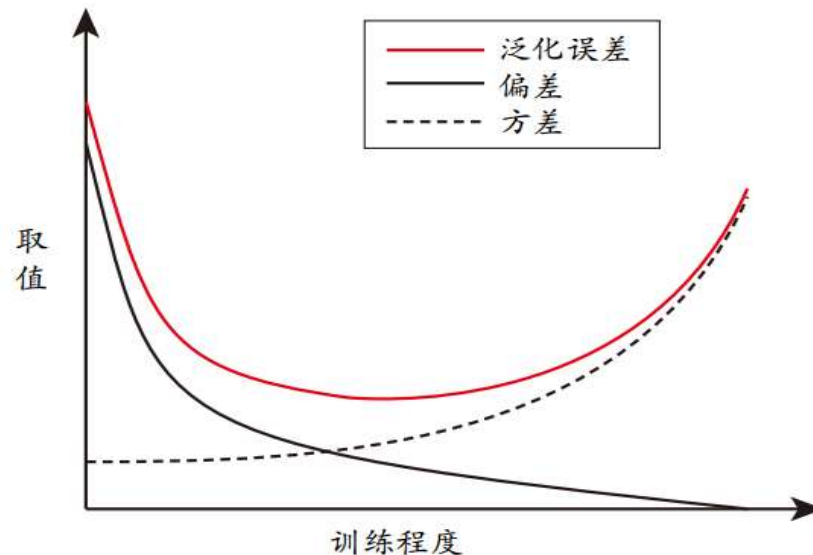
■ 偏差与方差

- ✓ 偏差度量了学习算法期望预测与真实结果的偏离程度；即刻画了学习算法本身的拟合能力；
 - ✓ 方差度量了同样大小训练集的变动所导致的学习性能的变化；即刻画了数据扰动所造成的影响；
 - ✓ 噪声表达了在当前任务上任何学习算法所能达到的期望泛化误差的下界；即刻画了学习问题本身的难度。
-
- ✓ 泛化性能是由学习算法的能力、数据的充分性以及学习任务本身的难度所共同决定的。给定学习任务为了取得好的泛化性能，需要使偏差小(充分拟合数据)而且方差较小(减少数据扰动产生的影响)。

■ 偏差与方差

□ 一般来说，偏差与方差是有冲突的，称为**偏差-方差窘境**。如右图所示，假如我们能控制算法的训练程度：

- ✓ 在训练不足时，学习器拟合能力不强，训练数据的扰动不足以使学习器的拟合能力产生显著变化，此时偏差主导泛化错误率；
- ✓ 随着训练程度加深，学习器拟合能力逐渐增强，方差逐渐主导泛化错误率；
- ✓ 训练充足后，学习器的拟合能力非常强，训练数据的轻微扰动都会导致学习器的显著变化，若训练数据自身非全局特性被学到则会发生过拟合。



泛化误差与偏差、方差的关系示意图



THANKS
祝学习快乐!